

Minneshantering på C:s vis

Eftersom detta är ett kompendium som behandlar C++ används operatorerna `new` och `delete` genomgående för att allokera och frigöra minne. Dessa operatorer är dock en nyhet som kommit i och med C++. I standard C använder man några funktioner för samma ändamål. Dessa funktioner är dock svårare att använda, och det rekommenderas att man använder C++:s motsvarigheter. Allmänt allokerar man minne enligt följande syntax:

```
datatyp * = (datatyp) malloc (antal_bytes);
```

Funktionen heter `malloc()`, och tar som parameter ett tal som berättar antalet bytes som skall allokeras. För att få reda på antalet bytes en viss datatyp upptar kan man använda `sizeof()`. Ett extra problem är att `malloc()` returnerar en `void *`, d.v.s. en pekare till ett minnesområde utan datatyp. Denna pekare måste typomvandlas (typecast) för att "passa" den pekare som skall användas. För att t.ex. allokera en `int` används följande:

```
int * Pekare = (int *)malloc ( sizeof (int) );
```

En hel del fulare än:

```
int * Pekare = new int;
```

Det är viktigt att storleken ges rätt, om den är för liten får man nästan garanterat problem då man försöker fylla minnet. Att allokera en `int` enligt nedanstående är helt fel:

```
int * Pekare = (int *)malloc ( 1 );
```

eftersom man då får endast en byte, och en `int` troligtvis behöver fyra. Allokering av vektorer kan man göra genom att multiplicera det antal bytes ett element i vektorn behöver med det antal element man vill ha. Allokering av 100 `char` kan göras på följande sätt:

```
char * Text = (char *) malloc ( sizeof (char) * 100 );
```

Alternativt kan man använda funktionen `calloc`, som tar två parametrar:

```
datatyp * = (datatyp) malloc (antal_element, bytes_per_element);
```

Samma vektor kan då allokeras på följande sätt:

```
char * Text = (char *) calloc ( 100, sizeof (char) );
```

Man bör även under C:s minneshantering frigöra allokerat minne. Detta görs med funktionen `free()`, som allmänt används enligt följande:

```
free (pekare);
```

För att frigöra t.ex. ovanstående `Text` kan man göra på följande sätt:

```
free ( Text );
```

Alla dessa funktioner finns definierade i headerfilen `stdlib.h`, så man behöver därför följande definition i början på varje fil som använder minnesallokering på C:s vis:

```
#include <stdlib.h>
```

[Förutgående](#)[Vanliga fel](#)[Hem](#)[Upp](#)[Nästa](#)[Kompilera C++-program](#)